

Dispensa Weekly Status Report

Week of 28 September – 2 October 2026

Prepared by Aref Haghgoorostami

Executive summary

AHEAD OF SCHEDULE

By the fifth day of an eleven-week plan, the roadmap items originally scheduled through week 5 (database, ingredients and suppliers CRUD, tests, CI, coverage) are merged and green, and authentication (week 6) is already underway as a draft PR.

Highlights

- Shipped a working backend skeleton: FastAPI + PostgreSQL 16 + Alembic, deployed in Docker Compose, with the first two resources (ingredients, suppliers) fully built as CRUD APIs with tenant scoping.
- Closed two coverage gaps and one real bug (a stale-read conflict message) found during manual code review, before either had a chance to reach production.
- Automated the whole quality gate: GitHub Actions now runs lint, type-check, tests and an 80% coverage floor on every push, with branch protection enforced even for the repo owner.
- Added structured JSON logging with a per-request ID, so every request is traceable end to end once the app is deployed.
- Started week 6 (authentication) three weeks ahead of schedule: register/login/JWT logic is drafted and pending three follow-up items before merge.

At a glance

15 PRs merged	31 / 31 Tests passing	30% Roadmap progress	0 Open blockers
-------------------------	---------------------------------	--------------------------------	---------------------------

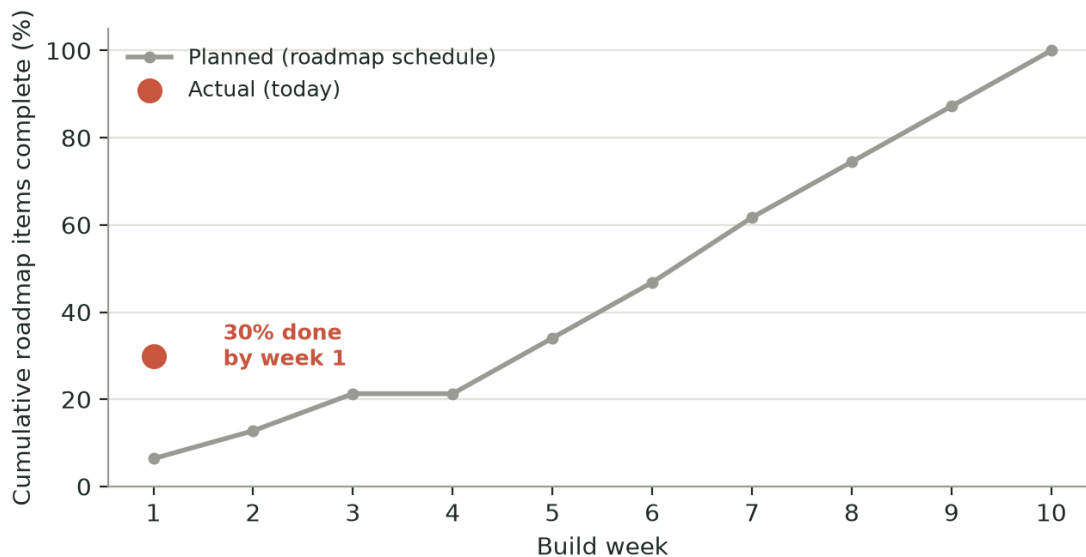
Progress vs plan

Work this week covered roadmap weeks 1, 2, 3 and 5 in full, plus the first draft of week 6. Week 4 is a planned exam block (no build items). Week 5's structured-logging item merged this week (PR #21) but the ROADMAP.md checkbox is still open and will be ticked in the next commit.

Planned (week)	Item	Status	Notes
Week 1	Create dispensa-api repo (README, ROADMAP, CLAUDE.md)	Done	—
Week 1	docs/DESIGN.md §1-4 (requirements, architecture, API, auth & tenancy)	Done	—
Week 1	GitHub profile: photo, bio, README; clean the iris repo	Not started	Issue #1, open
Week 2	docs/DESIGN.md §5-9 + design review	Done	Found 1 bug, 4 ambiguities during review
Week 2	Project skeleton: FastAPI, Ruff, mypy, pre-commit, /health	Done	—
Week 3	PostgreSQL in Docker Compose	Done	—
Week 3	SQLAlchemy models + first Alembic migration	Done	restaurants, users, ingredients, suppliers
Week 3	Ingredients CRUD	Done	—
Week 5	Test database, pytest fixtures	Done	3 weeks early

Planned (week)	Item	Status	Notes
Week 5	Ingredients + suppliers endpoint tests	Done	3 weeks early
Week 5	GitHub Actions CI (lint, type-check, tests)	Done	3 weeks early
Week 5	Coverage report, 80% gate in CI	Done	3 weeks early
Week 5	Structured JSON logs with a request ID	Done	Merged; checkbox not yet ticked
Week 6	users model, register + login	In progress	PR #22 draft, 3 items left

Cumulative roadmap progress: planned vs. actual



Tests-over-time trend is skipped this week — this is the first weekly report, so there is only one data point in docs/reports/metrics.json.

Delivered this week

15 PRs merged. Housekeeping PRs (ROADMAP checkbox ticks, two end-of-session docs/WORKLOG entries) are grouped at the end rather than given a full block each.

PR #4 — docs: add system design v1 github.com/Arefhg/dispensa-api/pull/4

What: Established the design everything else is built against: who uses Dispensa, the architecture, API conventions, auth/tenancy model and data model.

- docs/DESIGN.md §1–4 written by Aref; reviewed and one real bug caught before any code existed — the signed stock_movements.quantity column conflicted with a CHECK that only allowed positive values.
- Four design ambiguities resolved: correction semantics, report-month timezone, availability flooring, waste vs. negative stock.
- ROADMAP.md updated to move recipes & sales into the MVP scope (weeks 7–8), pushing the MVP date to 13 December.

Evidence: Design reviewed and merged; no automated tests (docs-only PR).

PR #5 / #6 — feat: project skeleton + chore: pre-commit

github.com/Arefhg/dispensa-api/pull/5, [/6](https://github.com/Arefhg/dispensa-api/pull/6)

What: Got a runnable, quality-gated FastAPI project in place before any business logic.

- Python 3.12 project, pyproject.toml (FastAPI, uvicorn, pytest, ruff, mypy), GET /health endpoint.
- .pre-commit-config.yaml wired to ruff-check, ruff-format and mypy; proved it actually blocks a bad commit (deliberately added an unused import, watched it get refused).
- mypy set to strict = true from the first file, not relaxed-then-tightened later.

Evidence: 1 test (GET /health); pre-commit verified to reject a bad commit locally.

PR #7 — feat: add PostgreSQL via Docker Compose and database connection

github.com/Arefhg/dispensa-api/pull/7

What: Gave the app a real, containerized database instead of an in-memory placeholder.

- docker-compose.yml: PostgreSQL 16, named volume, healthcheck, bound to 127.0.0.1 only.
- app/config.py (pydantic-settings) and app/db.py: sync SQLAlchemy engine/session with a naming-convention MetaData, so every constraint gets a predictable, greppable name.
- Alembic initialised with no models yet; DESIGN.md §10 updated with the sync-vs-async trade-off.

Evidence: No new tests (infra-only); migrations verified to run.

PR #8 — feat: add restaurants, users, ingredients models and first migration

github.com/Arefhg/dispensa-api/pull/8

What: Turned the data model from DESIGN.md §5 into real, constrained database tables.

- Typed SQLAlchemy 2 models (Mapped / mapped_column) for Restaurant, User, Ingredient, with CHECK constraints for role, unit and email lowercasing instead of Postgres ENUMs.
- First Alembic migration, generated then reviewed and fixed by hand before applying.
- Verified upgrade head → downgrade base → upgrade head all round-trip cleanly.

Evidence: Migration round-trip verified manually (no model-level tests yet at this point).

PR #9 — feat: add ingredients CRUD with tenant scoping and error envelope

github.com/Arefhg/dispensa-api/pull/9

What: First real business endpoint: owners can manage their ingredient list, and one restaurant can never see another's.

- Full CRUD (list/create/get/update/archive) in app/api + app/services, following the routers-call-services-not-business-logic split from CLAUDE.md.
- get_current_restaurant_id() introduced as the single, explicitly temporary stub for tenant identity — every router depends on it so it can be swapped for real JWT auth (week 6) without touching a single router.
- Standard {"error": {"code", "message"}} envelope for every error path; a real test database with per-test transaction rollback.
- Follow-up fix: DEV_RESTAURANT_ID now sourced from settings instead of a hardcoded constant, and refuses to run outside ENVIRONMENT=development.

Evidence: 11 tests, including a cross-tenant isolation test (restaurant A can't read/update/archive restaurant B's ingredient → 404).

PR #10 — docs: add WORKLOG.md and a session-end rule

github.com/Arefhg/dispensa-api/pull/10

What: Put a durable record of decisions and lessons in place so the project stays explainable as it grows.

- docs/WORKLOG.md created; CLAUDE.md updated with an explicit end-of-session rule to keep it current.

Evidence: Docs-only PR, no tests.

PR #15 — docs: plan e-invoice import (v1.1) github.com/Arefhg/dispensa-api/pull/15

What: Scoped the first post-MVP feature now, while the reasoning is fresh, without touching v1.0 code.

- DESIGN.md §11 added: owner uploads a FatturaPA XML invoice; a supplier_products table learns the ingredient/unit mapping once per supplier product.
- Post-MVP roadmap reordered: v1.1 e-invoice import ahead of v1.2 suggested orders and v1.3 alerts.

Evidence: Docs-only PR, no tests.

PR #16 — ci: add GitHub Actions CI workflow github.com/Arefhg/dispensa-api/pull/16

What: Every push and PR is now automatically linted, type-checked and tested — a broken change can no longer reach main unnoticed.

- .github/workflows/ci.yml: PostgreSQL 16 service container, ruff check, ruff format --check, mypy (scoped to match pre-commit exactly: app tests migrations scripts), pytest.
- Read-only token permissions, concurrency cancellation per ref, 15-minute timeout.
- Runner pinned to ubuntu-24.04 instead of ubuntu-latest, after CI's own annotation warned ubuntu-latest migrates to Ubuntu 26 on 2026-10-19.
- Branch protection enabled on main: required status check, strict (must be up to date), enforced for admins too — Aref can't merge a red PR via the UI either.

Evidence: CI run observed green both before and after the runner pin; README CI badge added.

PR #18 — feat: add suppliers CRUD github.com/Arefhg/dispensa-api/pull/18

What: Owners can now record which suppliers they buy from — the other half of "who delivered what."

- Suppliers CRUD reusing the ingredients pattern file-for-file (deps, error envelope, service/router shape, test structure).
- One new dependency: Pydantic's EmailStr + email-validator, with email lowercased by a field_validator and re-enforced at the DB level (CHECK email = lower(email)).
- Second Alembic migration, hand-reviewed; upgrade head → downgrade -1 → upgrade head verified.
- Fixed a real bug found by hand-review in both ingredients.py and suppliers.py: the 409 conflict message read the ORM object's .name after db.rollback(), which silently reloads the pre-update name — fixed to use the request data directly.

Evidence: 10 tests incl. cross-tenant isolation and an email-lowercasing regression test guarding the 409-message fix.

PR #19 — test: add coverage measurement, CI gate at 80%

github.com/Arefhg/dispensa-api/pull/19

What: Makes untested code visible and keeps the test floor from eroding as the codebase grows.

- pytest-cov added; coverage measured for app/, gated at 80% in CI only via `--cov-fail-under=80` (not in local runs, so an in-progress feature's temporary dip doesn't interrupt the dev loop).
- A 'Coverage summary' CI step writes the report into the GitHub job summary even when the gate fails.
- Closed two real coverage gaps (97% → 99%): `get_current_restaurant_id()`'s success path and its `DEV_RESTAURANT_ID-not-set` guard.
- No coverage badge or number in README, deliberately — both risk going stale; the CI job summary is the live source.

Evidence: 2 new gap-closing tests; coverage now 98.97% against an 80% floor.

PR #21 — feat: add structured JSON logs with a request ID

github.com/Arefhg/dispensa-api/pull/21

What: Every request is now traceable end to end by one ID, in a machine-parseable format ready for log aggregation once deployed.

- `app/logging_config.py`: `JsonFormatter` emits one JSON object per log line (time, level, message, logger, request_id, plus any extra fields); never logs secrets.
- `app/middleware.py`: `RequestIdMiddleware` reuses a valid incoming X-Request-ID header or generates a UUID, logs exactly one structured access line per request, and attaches the ID to the response header.
- Uvicorn's own access/error loggers rerouted through the same JSON handler instead of logging plain text separately.
- `configure_logging()` called from FastAPI's lifespan hook, not at import time, so uvicorn's own logging setup doesn't silently undo it.

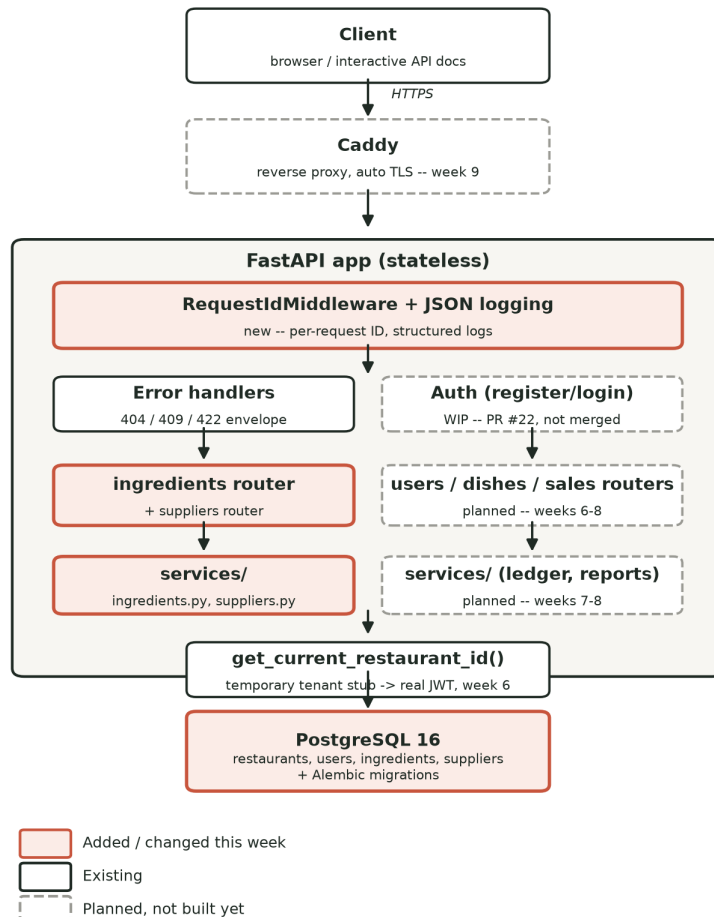
Evidence: 7 new tests: formatter with/without exceptions, response header present, valid/invalid incoming request IDs, exactly-one access line, request ID preserved through an unhandled exception.

Housekeeping PRs merged this week

- PR #14 — chore: tick ROADMAP checkboxes for weeks 1–3 build items already merged but never checked off; strengthened the CLAUDE.md end-of-session rule.
- PR #17 / #20 — docs: session-close entries (`docs/WORKLOG.md`), ticking ROADMAP boxes for the day's work.

Architecture snapshot

Request flow stays as designed in `DESIGN.md` §2: client → Caddy (reverse proxy, not built until week 9) → the stateless FastAPI app → PostgreSQL. This week's additions sit inside the app: request-ID logging middleware, the ingredients and suppliers routers/services, and the first two migrations against Postgres.



Technical decisions

From DESIGN.md §10 and docs/WORKLOG.md — decisions made this week only.

Decision	Why	Trade-off
Ledger instead of stored stock	Every stock number stays explainable from its movement history; nothing is silently overwritten.	Current stock requires a SUM on read; needs good indexes.
Recipes & sales moved into the MVP (from v1.1)	Staff's main job is recording sales; without recipes the staff role would be almost empty.	~1 extra build week; MVP target moved to 13 December.
Negative stock allowed	A real sale or waste record must never be rejected just because it makes the number negative.	Owner's stock report can show negative numbers; availability is floored at 0 for staff.
Shared database + restaurant_id column	Much simpler to run and migrate than a database per tenant, at this scale.	Isolation depends on every query filtering correctly — mitigated by one shared helper and dedicated tests.
Sync SQLAlchemy, not async	Simpler code and tests; sufficient at Dispensa's target scale (~100 restaurants).	Would need to switch to async if measured load ever requires it.
role / unit as String + CHECK, not Postgres ENUM	Simpler to extend later.	No enum-alteration edge cases to manage.

Decision	Why	Trade-off
<code>get_current_restaurant_id()</code> as the single temporary tenant stub	Every router depends on it, so swapping in real JWT auth (week 6) touches one function, not every router.	Explicitly temporary; must be removed once auth lands.
Coverage gate (80%) enforced in CI only, not local runs	Local pytest just reports the number — a dipping-mid-feature coverage number doesn't interrupt the dev loop.	Local runs can silently fall under 80% between commits.
<code>enforce_admins</code> : true on branch protection	Removes Aref's own ability to bypass a red check via the merge button, not just other contributors'.	No manual override is possible, even for a trivial hotfix.

Risks and issues

Risk	Impact	Mitigation	Owner
Auth PR (#22) merges mid-feature	Week 6 roadmap items (protected endpoints, <code>/me</code> , owner-only actions, isolation tests, rate limiting) are all blocked on it.	3 known follow-ups already scoped in the PR description (switch to HTTPBearer, broaden argon2 exception handling, write <code>tests/test_auth.py</code>); resuming next session.	Aref / Claude
GitHub profile cleanup not started	Cosmetic only; no effect on the API.	Scheduled as a short standalone task; tracked as issue #1.	Aref

Problems solved this week

- 409 conflict message showed the stale name. `db.rollback()` expires SQLAlchemy ORM attributes, so reading `ingredient.name` after a rollback silently re-fetched the pre-update name from the database. Fixed to build the message from the request data directly; regression tests added for both ingredients and suppliers.
- Every local database command took ~90s extra on Windows. `localhost` in `DATABASE_URL` resolved to `IPv6 ::1` first, which nothing was bound to. Fixed by using `127.0.0.1` explicitly everywhere.
- `mypy --strict` flagged `Settings()` as missing a required argument. It couldn't see that `pydantic-settings` fills fields from the environment at runtime. Fixed by enabling the `pydantic.mypy` plugin, not by suppressing with `type: ignore`.

Next week

#	Deliverable	Goal	Expected outcome
1	Finish and merge auth (PR #22)	Switch to HTTPBearer, handle argon2's remaining exception types, write <code>tests/test_auth.py</code> covering register/login/token edge cases.	Register, login and <code>/me</code> merged and tested; week 6 foundation in place.
2	users model: protected endpoints, owner-only actions	Owner can add/list/deactivate staff; endpoints require a valid token.	Role-based authorization enforced on every relevant endpoint.

#	Deliverable	Goal	Expected outcome
3	Tenant isolation tests + login rate limiting	Prove restaurant A can never read or act on restaurant B's data under real auth; block brute-force login attempts.	Week 6 'done when' criteria met — isolation tests pass.
4	GitHub profile cleanup (issue #1)	Photo, bio, profile README; clean the iris repo.	Issue #1 closed.

Support needed

None blocking this week. One thing worth a quick sign-off next session: PR #22 requires the JWT secret to be ≥ 32 characters in every environment (not just outside development, as originally planned) — simpler to implement and our .env/CI values already meet it, but flagging since it wasn't the originally discussed rule.

Appendix

Commands and tools used

- `pytest --cov-fail-under=80` — ran the full suite with coverage (31 passed, 98.97% coverage)
- `ruff check . / ruff format --check .` — lint and formatting
- `mypy app tests migrations scripts` — static type checking (strict mode)
- `alembic upgrade head / downgrade` — database migrations, verified round-trip
- `docker compose up -d` — local PostgreSQL 16
- `git log`, GitHub MCP tools (`list_pull_requests`, `pull_request_read`) — source of this report's PR data
- `reportlab + matplotlib` — this PDF and its charts

Glossary

Term	Meaning
Ledger	An append-only record of every change (here: stock movements); current totals are always a sum of history, never an edited number.
Multi-tenancy / isolation	Many restaurants sharing one system, with data isolated per restaurant (<code>restaurant_id</code> on every table) so none can read or change another's data.
Migration (Alembic)	A versioned, reversible change to the database schema, stored in git like code.
CI / Coverage	Automated checks (lint, type-check, tests) that run on every push/PR; coverage is the % of app code exercised by the test suite.
JWT	JSON Web Token — a signed token carrying the logged-in user's identity, sent with each request.
Argon2	A password-hashing algorithm designed to be slow, to resist brute-force attacks.